

© Joshua M. Willman 2020

J. M. Willman, *Beginning PyQt*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-5857-6_8

8. Working with the Clipboard

Joshua M. Willman¹

(1) Hampton, VA, USA

One of the major benefits of GUIs is the ability to create programs that can visually interact with your system and other applications. Previous chapters have shown how to use dialog boxes to open and save files to your computer, change fonts or colors, and send images to a printer. In Chapter 8 you will see how to use the clipboard and drag and drop functions to extend the capabilities of your programs even further.

The **clipboard** is a location in your computer's memory that is used to temporarily store data that you have copied or cut from an application. The clipboard can store a number of different types of data, including text, images, gifs, and more. Information that is stored on your system's clipboard can be pasted into other applications, as long as it knows how to work with the type of data stored in the clipboard.

QClipboard is a PyQt class that is used to interact with your system's clipboard, allowing you to copy and paste between GUIs.

Chapter 8 introduces you to

- The `QClipboard` and `QMimeData` classes
- The Drag and Drop system in PyQt
- The `QListWidget` class for displaying item-based lists

The QClipboard Class

The **QClipboard** class makes your system's clipboard available so that you can copy and paste data such as text, images, and HTML text between applications. Qt widgets that can be used to manipulate textual information, such as `QLineEdit`, `QTextEdit`, and `QListWidget`, support using the clipboard. If you want to paste an image from the clipboard into an application, be sure to use widgets that support graphics, such as `QLabel`.

Including the clipboard in your project is pretty straightforward in PyQt. In order to access the `QClipboard` object in an application, create an instance of the clipboard by

```
clipboard = QApplication.clipboard()
```

To see one way to retrieve an image that has been copied to the clipboard, take a look at the following code:

```
self.label = QLabel() # Create label to hold image
self.clipboard = QApplication.clipboard() # Create cb object
self.label.setPixmap(self.clipboard.pixmap())
```

The process for text or HTML text is similar. Another way to get data is to use the `QMimeData` class which will be covered in the “Explanation” section of this example.

The events that occur between your system and an application built using PyQt are handled by `QApplication`. The `clipboard` gives you the ability to send or receive data in your application. This means that you can not only get data from other programs but can also send it out, as well. However, the clipboard can only hold one object at a time. So if you copy an image to the clipboard and then copy text, only the text will be available and the image will have been deleted.

In Listing 8.1 you will see how to set up the clipboard and actually be able to visualize its contents after copying text from another window.

```
# clipboard_ex.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow, QPushButton, QTextEdit, QDockWidget, QVBoxLayout)
from PyQt5.QtCore import Qt, QSize

class ClipboardEx(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen
        """
        self.setMinimumSize(QSize(500, 300))
        self.setWindowTitle("Clipboard Example")
        self.central_widget = QTextEdit()
        self.setCentralWidget(self.central_widget)
        self.createClipboard()
        self.show()
    def createClipboard(self):
        """
        Set up clipboard and dock widget to display text from
        the clipboard.
        """
        # Create dock widget
        clipboard_dock = QDockWidget()
        clipboard_dock.setWindowTitle("Display Clipboard Contents")
        clipboard_dock.setAllowedAreas(Qt.TopDockWidgetArea)
        dock_frame = QFrame()
        self.cb_text = QTextEdit()
        paste_button = QPushButton("Paste")
        paste_button.clicked.connect(self.pasteText)
        dock_v_box = QVBoxLayout()
        dock_v_box.addWidget(self.cb_text)
        dock_v_box.addWidget(paste_button)
        # Set the main layout for the dock widget,
        # then set the main widget of the dock widget
        dock_frame.setLayout(dock_v_box)
        clipboard_dock.setWidget(dock_frame)
        # Set initial location of dock widget
```

```

self.addDockWidget(Qt.TopDockWidgetArea, clipboard_dock)
# Create instance of the clipboard
self.clipboard = QApplication.clipboard()
self.clipboard.dataChanged.connect(self.copyFromClipboard)
def copyFromClipboard(self):
    """
    Get the contents of the system clipboard.
    """
    mime_data = self.clipboard.mimeData()
    if mime_data.hasText():
        self.cb_text.setText(mime_data.text())
        #self.cb_text.repaint() # Uncomment if text not updating
def pasteText(self):
    """
    Paste text from the clipboard if button is clicked.
    """
    self.central_widget.paste()
    #self.central_widget.repaint() # Uncomment if text not updating
if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = ClipboardEx()
    sys.exit(app.exec_())

```

Listing 8-1 Example code that demonstrates how to use the clipboard

The finished application can be seen in Figure 8-1. The top text edit widget displays the current contents of the clipboard. The user can then paste it into the main window which is the lower text edit widget.

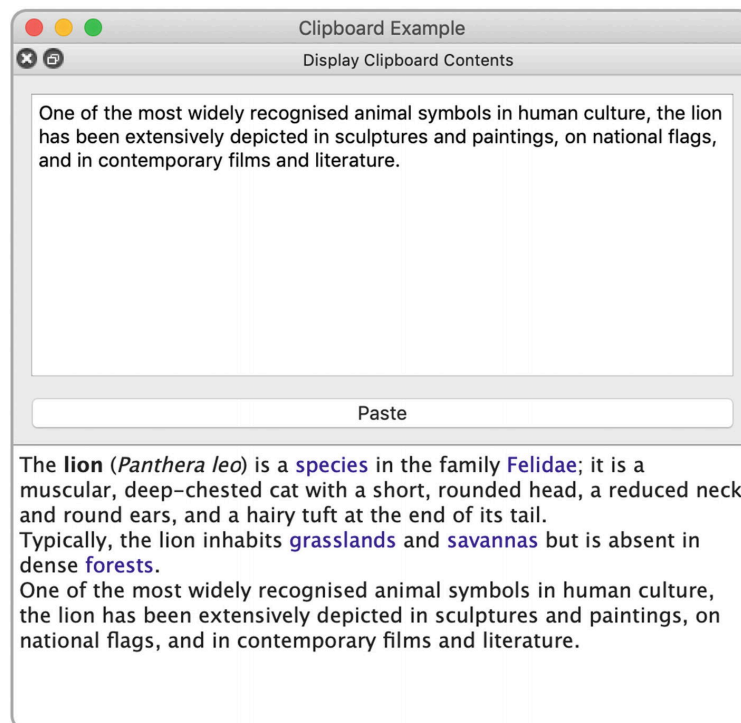


Figure 8-1 Example GUI that allows the user to see the contents of the clipboard in the top dock widget

Explanation

In some applications you may actually want to see the contents of the clipboard in a separate window before pasting it into the main window. A dock widget, especially one that can float separate from the main window, is perfect to use as a clipboard manager.

After importing classes and setting up the window, we set the `central_widget` of the main window to be a `QTextEdit` widget. The central widget is where we will edit the text that is pasted from the clipboard.

The `clipboard_dock` widget is set up so that it can either float or be attached to the top of the main window. We need to use a `QFrame` container to hold the `cb_text` and `paste_button` widgets. If new text is copied from another application, then `cb_text` will display the text. If the user wants to retain the text, then they can paste it into the `central_widget`.

Using a single line, PyQt makes it simple to include the clipboard in an application.

```
self.clipboard = QApplication.clipboard() # Create the clipboard object
self.clipboard.dataChanged.connect(self.copyFromClipboard)
```

The `dataChanged()` method emits a signal if the contents of the clipboard have changed. If a change has occurred, then the `cb_text` widget is updated to display the new clipboard text using the `copyFromClipboard()` method. To check what kind of data is stored in the clipboard, we use the `QMimeData` class.

The **QMimeData** class is used for both the clipboard and the drag and drop system in PyQt, the latter of which will be introduced later in this chapter. The Multipurpose Internet Mail Extensions (MIME) format supports not only text but HTML, URLs, images, and color data, as well. Objects created from the `QMimeData` class ensure that information can be safely moved between applications and also between objects in the same application.

```
mime_data = self.clipboard.mimeData()
```

The method `mimeData()` returns information about the data currently in the clipboard. To check if the object can return plain text, we use the `hasText()` method. If the data is text, then we get the text using `mime_data.text()` and set the text of the `QTextEdit` widget using `setText()`. A similar process is also used to access other kinds of data using `QMimeData`.

Finally, the `QTextEdit` method `paste()` is called in `pasteText()` to fetch the text in the clipboard if the button is pressed. The `repaint()` method is used to force the text of the widget to update.

Project 8.1 – Sticky Notes GUI

Sometimes you have an idea, a note, or a bit of information that you need to quickly jot down. Maybe you need to remind yourself of an appointment and need to make a note to yourself. You only need a small, temporary, maybe even colorful, area to help brainstorm and organize those ideas. Sticky notes are perfect for those uses and more.

The sticky notes GUI, shown in Figure 8-2, allows you to open as many windows as you want. You can edit the text of each note individually, change the color of a note, and also paste text from the clipboard. This project demonstrates

a practical use for the clipboard class and acts as a foundation if you choose to build your own sticky notes application.

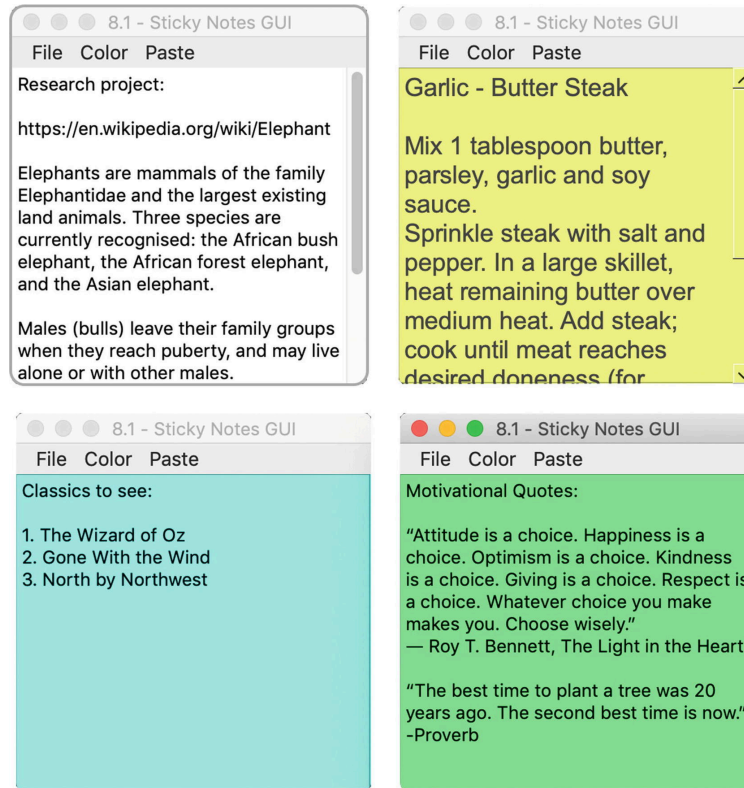


Figure 8-2 The sticky notes GUI

Sticky Notes GUI Solution

The sticky notes GUI is relatively simple, created of a single `QTextEdit` widget that serves as the central widget of the main window (Listing 8-2). The menu system allows you to create a new note, clear the text in the text edit widget, quit the application, change the background color, and paste text from the clipboard.

```
# stickynotes.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow, QAction, QTextEdit)
from PyQt5.QtCore import QSize

class StickyNotes(QMainWindow):
    # Static variables
    note_id = 1
    notes = []

    def __init__(self, note_ref=str()):
        super().__init__()
        self.note_ref = note_ref
        StickyNotes.notes.append(self)
        self.initializeUI()

    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen
        """
        self.setMinimumSize(QSize(250, 250))
        self.setWindowTitle("8.1 - Sticky Notes GUI")
        self.central_widget = QTextEdit()
        self.setCentralWidget(self.central_widget)
```

```

self.createMenu()
self.createClipboard()
self.show()
def createMenu(self):
    """
    Create simple menu bar and menu actions
    """
    # Create actions for the file menu
    self.new_note_act = QAction('New Note', self)
    self.new_note_act.setShortcut('Ctrl+N')
    self.new_note_act.triggered.connect(self.newNote)
    self.close_act = QAction('Clear', self)
    self.close_act.setShortcut('Ctrl+W')
    self.close_act.triggered.connect(self.clearNote)
    self.quit_act = QAction('Quit', self)
    self.quit_act.setShortcut('Ctrl+Q')
    self.quit_act.triggered.connect(self.close)
    # Create actions for the color menu
    self.yellow_act = QAction('Yellow', self)
    self.yellow_act.triggered.connect(lambda: self.changeBackground(self.yellow_act.text()))
    self.blue_act = QAction('Blue', self)
    self.blue_act.triggered.connect(lambda: self.changeBackground(self.blue_act.text()))
    self.green_act = QAction('Green', self)
    self.green_act.triggered.connect(lambda: self.changeBackground(self.green_act.text()))
    # Create actions for the paste menu
    self.paste_act = QAction('Paste', self)
    self.paste_act.setShortcut('Ctrl+V')
    self.paste_act.triggered.connect(self.pasteToClipboard)
    # Create menubar
    menu_bar = self.menuBar()
    menu_bar.setNativeMenuBar(False) # Uncomment to display menu in window on MacOS
    # Create file menu and add actions
    file_menu = menu_bar.addMenu('File')
    file_menu.addAction(self.new_note_act)
    file_menu.addAction(self.close_act)
    file_menu.addAction(self.quit_act)
    # Create color menu and add actions
    file_menu = menu_bar.addMenu('Color')
    file_menu.addAction(self.yellow_act)
    file_menu.addAction(self.blue_act)
    file_menu.addAction(self.green_act)
    # Create paste menu and add actions
    file_menu = menu_bar.addMenu('Paste')
    file_menu.addAction(self.paste_act)
def createClipboard(self):
    """
    Set up clipboard.
    """
    self.clipboard = QApplication.clipboard()
    self.clipboard.dataChanged.connect(self.copyToClipboard)
def newNote(self):
    """
    Create new instance of StickyNotes class.
    """
    self.note_ref = str("note_%d" % StickyNotes.note_id)
    StickyNotes().show()
    StickyNotes.note_id += 1
def clearNote(self):
    """
    Delete the current note's text.
    """

```

```

        self.central_widget.clear()
    def copyToClipboard(self):
        """
        Get the contents of the system clipboard.
        """
        return self.clipboard.text()
    def pasteToClipboard(self):
        """
        Get the contents of the system clipboard and paste
        into the note.
        """
        text = self.copyToClipboard()
        self.central_widget.insertPlainText(text + '\n')
    def changeBackground(self, color_text):
        """
        Change a note's background color.
        """
        if color_text == "Yellow":
            self.central_widget.setStyleSheet("background-color: rgb(248, 253, 145)")
        elif color_text == "Blue":
            self.central_widget.setStyleSheet("background-color: rgb(145, 253, 251)")
        elif color_text == "Green":
            self.central_widget.setStyleSheet("background-color: rgb(148, 253, 145)")
if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = StickyNotes()
    sys.exit(app.exec_())

```

Listing 8-2 Code for sticky notes GUI

The completed project can be seen in Figure [8-2](#).

Explanation

The sticky notes GUI is a good project to introduce the concept of **single-document interface (SDI)**. SDI is a method that organizes GUIs into individual windows that are handled separately. Even though the sticky notes application allows you to create multiple instances of the GUI at the same time, each window is separate and independent from the others. The contrast is **multiple-document interface (MDI)**, where a single parent window contains and controls multiple nested child windows. An example of MDI can be found in Chapter [12](#).

We create the `StickyNotes` class for the GUI and first include two static variables, `note_id`, used to give a unique name to each new window, and `notes`, to keep track of the many different windows by appending them to a list. The static variables are shared by all instances of the class.

The menu system is set up just like in previous chapters. The `Color` menu allows the user to select a background color for each note. If the user wants to paste text from the clipboard into a widget, they can either use the `Paste` menu entry or the hot key `Ctrl+V`.

Let's take a look at the different class methods. The `clipboard` object is created and updated using the `dataChanged()` signal in `createClipboard()`. Each new note is given a new name, `note_ref`, when they are created based on the current `note_id` value. The other functions allow

you to interact with the text and clipboard or edit the background color of the `central_widget`.

Drag and Drop in PyQt

The drag and drop mechanism allows a user to perform tasks in a GUI by selecting items, such as icons or images, and move them into another window or onto another object. PyQt also makes including this behavior in an application very simple, as well. To allow widgets to have drag and drop functionality, you only need to set their `setAcceptDrops()` and `setDragEnabled()` properties to `True`.

With drag and drop functionality enabled, you can move items from one text edit, list, or table object to another in PyQt. `QMimeData` can also be used to handle custom data types.

Listing 8-3 illustrates how to drag and drop icons between two `QListWidget` objects in the same GUI window.

```
# drag_drop.py
# Import necessary modules
import sys, os
from PyQt5.QtWidgets import (QApplication, QWidget, QListWidget, QLabel, QGridLayout, QListWidgetItem)
from PyQt5.QtCore import QSize
from PyQt5.QtGui import QIcon
class DragAndDropGUI(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen
        """
        self.setGeometry(100, 100, 500, 300)
        self.setWindowTitle("Drag and Drop Example")
        self.setupWidgets()
        self.show()
    def setupWidgets(self):
        """
        Create and arrange widgets in window
        """
        icon_label = QLabel("ICONS", self)
        icon_widget = QListWidget()
        icon_widget.setAcceptDrops(True)
        icon_widget.setDragEnabled(True)
        icon_widget.setViewMode(QListWidget.IconMode)
        image_path = "images"
        for img in os.listdir(image_path):
            list_item = QListWidgetItem()
            list_item.setText(img.split(".")[0])
            list_item.setIcon(QIcon(os.path.join(image_path,
                                                  "{0}".format(img))))
            icon_widget.setIconSize(QSize(50, 50))
            icon_widget.addItem(list_item)
        list_label = QLabel("LIST", self)
        list_widget = QListWidget()
        list_widget.setAlternatingRowColors(True)
        list_widget.setAcceptDrops(True)
```

```
list_widget.setDragEnabled(True)
# Create grid layout
grid = QGridLayout()
grid.addWidget(icon_label, 0, 0)
grid.addWidget(list_label, 0, 1)
grid.addWidget(icon_widget, 1, 0)
grid.addWidget(list_widget, 1, 1)
self.setLayout(grid)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = DragAndDropGUI()
    sys.exit(app.exec_())
```

Listing 8-3 Code that demonstrates an example of drag and drop

The drag and drop GUI can be seen in Figure 8-3.

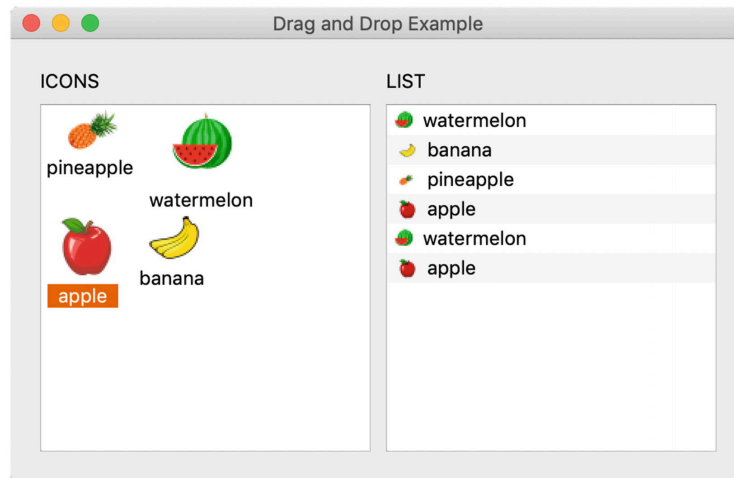


Figure 8-3 Two QListWidget objects used to demonstrate drag and drop

Explanation

There are two instances of the `QListWidget` class created, `icon_widget` and `list_widget`. The `icon_widget` object displays objects in `IconMode`, set by

```
icon_widget.setViewMode(QListWidget.IconMode)
```

The `QListWidget` default setting shows items in a list. To set up the drag and drop capability for `icon_widget`, call the `setAcceptDrops()` and `setDragEnabled()` methods. Then repeat the process for `list_widget`.

```
icon_widget = QListWidget()
icon_widget.setAcceptDrops(True)
icon_widget.setDragEnabled(True)
```

When one of the icons that are loaded into the `icon_widget` is dragged onto the `list_widget`, the list updates its contents to include the new item. Dropping an item from one `QListWidget` to the other adds a new item to that list.

The next section introduces the `QListWidget` class and a few of its methods.

The QListWidget Class

The **QListWidget** class creates a widget with an item-based interface that makes it simpler for adding and removing items. Items can be added either when the widget is created in code or added on later. The **QListWidgetItem** class is used in conjunction with **QListWidget** to serve as an item that can be used with the list. In the previous example, `list_item` creates an item that includes text and an icon to be added to the list using the `addItem()` method.

Listing 8-4 briefly demonstrates how to add, insert, remove, and clear all items from a **QListWidget**.

```
# listwidget_ex.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QListWidget, QPushButton, QHBoxLayout, QVBoxLayout,
class GroceryListGUI(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen
        """
        self.setGeometry(100, 100, 400, 200)
        self.setWindowTitle("QListWidget Example")
        self.setupWidgets()
        self.show()
    def setupWidgets(self):
        """
        Create and arrange widgets in window
        """
        self.list_widget = QListWidget()
        self.list_widget.setAlternatingRowColors(True)
        # Initialize the QListWidget with items
        grocery_list = ["grapes", "broccoli", "garlic", "cheese", "bacon", "eggs", "waffles", "rice", "
        for item in grocery_list:
            list_item = QListWidgetItem()
            list_item.setText(item)
            self.list_widget.addItem(list_item)
        # Create buttons
        add_button = QPushButton("Add")
        add_button.clicked.connect(self.addListItem)
        insert_button = QPushButton("Insert")
        insert_button.clicked.connect(self.insertItemInList)
        remove_button = QPushButton("Remove")
        remove_button.clicked.connect(self.removeOneItem)
        clear_button = QPushButton("Clear")
        clear_button.clicked.connect(self.list_widget.clear)
        # Create layout
        right_v_box = QVBoxLayout()
        right_v_box.addWidget(add_button)
        right_v_box.addWidget(insert_button)
        right_v_box.addWidget(remove_button)
        right_v_box.addWidget(clear_button)
        left_h_box = QHBoxLayout()
        left_h_box.addWidget(self.list_widget)
        left_h_box.addLayout(right_v_box)
```

```

self.setLayout(left_h_box)
def addListItem(self):
    """
    Add a single item to the list widget.
    """
    text, ok = QInputDialog.getText(self, "New Item", "Add item:")
    if ok and text != "":
        list_item = QListWidgetItem()
        list_item.setText(text)
        self.list_widget.addItem(list_item)
def insertItemInList(self):
    """
    Insert a single item into the list widget under the
    current highlighted row.
    """
    text, ok = QInputDialog.getText(self, "Insert Item", "Insert item:")
    if ok and text != "":
        row = self.list_widget.currentRow()
        row = row + 1 # select row below current row
        new_item = QListWidgetItem()
        new_item.setText(text)
        self.list_widget.insertItem(row, new_item)
def removeOneItem(self):
    """
    Remove a single item from the list widget.
    """
    row = self.list_widget.currentRow()
    self.list_widget.takeItem(row)
if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = GroceryListGUI()
    sys.exit(app.exec_())

```

Listing 8-4 An example of using the QListWidget class to manage items in a list

Your GUI should look similar to the one in Figure [8-4](#).

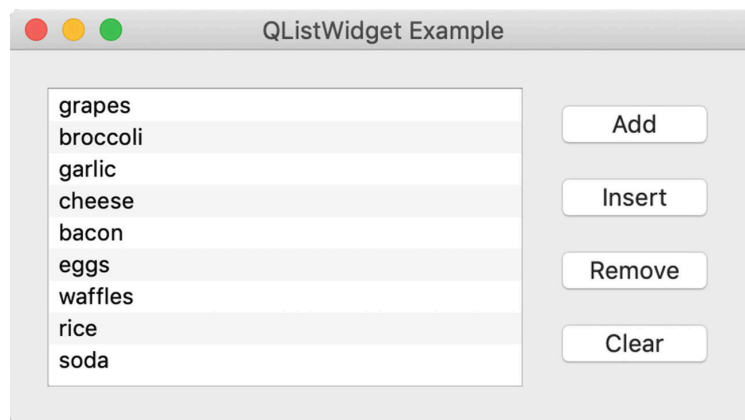


Figure 8-4 QListWidget could be used to display objects in an inventory or items in a directory

Explanation

QListWidget allows you to manage the items displayed in the GUI window. For alternating row colors, set the `setAlternatingRowColors()` methods to `True`.

The `list_widget` object is populated with items from the list when the program begins. Using the buttons on the right, the user can edit the list items.

Each method in the `GroceryListGUI` class is used to demonstrate one of the following `QListWidget` methods :

- `addItem()` – Add an item to the end of a list
- `insertItem()` – Insert an item at the specified row
- `takeItem()` – Remove an item from the specified row
- `clear()` – Remove all items from the list

Summary

The `QClipboard` class allows GUI applications to receive and send data from the system's clipboard. Drag and drop is another type of functionality that GUIs can utilize to pass data between widgets and other programs. Drag and drop is very simple to include in your own projects with only a few lines of code. The `QMimeData` class handles various kinds of data types for both clipboard and drag and drop systems, ensuring proper data handling.

Many of PyQt's widgets for editing text already include the ability to interact with the clipboard, so you won't often need to include the code for the clipboard in your program.

In the next chapter, we will see how to implement animation and graphics into PyQt applications and learn how to create custom widgets.